

INSTITUT POLYTECHNIQUE DE PARIS

Master DataAI

M1 Internship Report

Radio Map Prediction: Dataset Construction Pipeline and Training Framework Refactoring

Author:

Chenxin LUO

First-Year Master Student

Internship Supervisor:

Shanshan WANG (Télécom Paris)

Academic Evaluator:

Olivier FERCOQ (Télécom Paris)

PhD Mentor:

Shuangning LI (Télécom Paris)

Palaiseau, France

July 2, 2026

Contents

1	Introduction	1
1.1	General Context and Background	1
1.2	Problem Statement	1
1.2.1	Benchmark Dataset Constraints	1
1.2.2	Hard-scalable Training Framework	2
1.3	Core Contributions	2
1.4	Report Organization	3
2	Technical Background and Related Work	5
2.1	Radio Map Estimation and Deep Learning	5
2.2	NVIDIA Sionna Platform	6
2.3	Mathematical Formulation	7
2.3.1	Generalized Domain and Configurable Tensor Definitions	7
2.3.2	Configurable Learning Paradigms and Loss Formulations	8
2.3.3	First-Order Optimization	9
2.3.4	Physical Meaning of Training Error	9
3	Scalable Data Pipeline and Dataset Construction	11
3.1	Overview of the 7-Stage Architectural Data Pipeline	11
3.2	The Sliding-Window Slicing Mechanic	12
3.3	Sionna Scene Files Generation	14
3.4	City Map Rasterization and Transmitter Matrix Generation	15
3.5	Transmitter Relocation	16
3.6	Multi-Source Radio Map Matrix Generation	17
3.7	Comparison with RadioMapSeer	17
4	Training Framework Refactoring	19
4.1	Legacy Code Evaluation and Technical Debt	19
4.2	Modular Object-Oriented Framework Design	19
4.2.1	Template Method Pattern and Subclass Hooks	20
4.3	YAML-Driven Configuration and Cluster Deployment	21
4.4	Experiment Tracking	21
4.4.1	TensorBoard Monitoring	21
4.4.2	Log File Management	22
4.4.3	Automated Experiment Result Persistence	22
5	Experimental Evaluation and Functional Verification	23
5.1	Dataset Validation	23
5.2	Experimental Training Dynamics and Record Validation	25

6	Conclusion and Future Work	27
6.1	Summary of Internship Achievements	27
6.2	Future Research Outlook	27

Chapter 1

Introduction

1.1 General Context and Background

With the rapid development of wireless communication technologies, there has been an unprecedented increase in the number of base stations (BSs) in urban areas. Consequently, characterizing signal strength across a given spatial area—creating a **Radio Map**—has become a crucial prerequisite to protect public health and safety through the assessment of radio-frequency electromagnetic field (RF-EMF) exposure [1].

For a long time, the construction of high-fidelity radio maps has relied primarily on computationally intensive physics-based models, most notably GPU-accelerated ray tracing methods [1]. To further accelerate this process, modern ray tracing techniques have adopted Bounding Volume Hierarchies (BVH) [2], a concept derived from computer graphics. However, despite these geometric optimizations, the total computational complexity still rises sharply in urban environments with complex spatial structures. This high online computational overhead limits physics-driven models to static, offline planning scenarios, rendering them unsuitable for applications requiring real-time network orchestration or dynamic exposure monitoring.

To further reduce computational overhead, deep learning (DL) has been introduced, decomposing the radio map estimation problem into offline computation and online application phases. Taking the RadioUNet framework as an example, radio map estimation is treated as an **image-to-image** translation task [3]. Spatial information is input into the neural network as images; during the offline training phase, this information is correlated with the target radio maps, and this relationship is encoded within the network’s parameters, such as its weights. Once training converges, the online application phase requires only a single, rapid forward pass to generate a high-precision radio map within milliseconds.

1.2 Problem Statement

Although the classic RadioUnet method performs impressively as a baseline for radio map estimation, the dataset and training framework it employs impose significant limitations when attempting to extend the method.

1.2.1 Benchmark Dataset Constraints

The performance of any deep learning model is fundamentally constrained by the distribution of its underlying training data. As highlighted in comprehensive surveys [4], the vast majority of publicly available radio map datasets—such as the standard RadioMapSeer benchmark—are

strictly limited to synthetic geometric structures, and most contain only samples featuring single, isolated transmitters.

However, in practical application scenarios, multiple transmitters are often present [5]. Based on the assumption of incoherent propagation, the total received power at a given location can be obtained by linearly summing the received power from each transmitter [6], thereby enabling the evaluation of the radio map.

If the training samples contain only a single transmitter, the model will fail to effectively learn the characteristics of radio maps in multi-source scenarios; similarly, synthetic spatial feature maps may not cover certain complex scenarios found in real-world cities. Therefore, constructing an automated pipeline to efficiently create a large-scale, high-fidelity dataset—leveraging real-world city maps and statistical records of transmitters—is of significant value.

1.2.2 Hard-scalable Training Framework

Beyond the limitations of the datasets, current training frameworks for neural networks addressing the radio map estimation problem are difficult to scale. These frameworks often rely on procedural programming and are built upon interactive Jupyter Notebooks for rapid prototyping. In practice, this manifests as extensive code duplication, tight coupling between data loading and training logic, and a reliance on numerous hard-coded configuration parameters. In details, there are several issues:

- **I/O Inefficiency:** Take RadioMapSeer as an example, it stores vast amounts of spatial feature data in a loose image format, which creates a severe disk read bottleneck during data loading and slows down training.
- **Scalability and Deployment Limitation:** The large amount of procedural code prevents the reuse of training logic when switching between different neural network architectures; extensive hard-coding of configuration parameters necessitates digging through the code to locate and modify settings, making it difficult to ensure consistency; furthermore, reliance on interactive execution methods makes it extremely difficult to deploy training tasks to remote high-performance computing clusters.
- **Lack of Experiment Tracking:** The training code relies heavily on manually printing various parameters and matrix changes to the command line to monitor the training process. This approach lacks both intuitiveness and a record-keeping mechanism, making it difficult to quickly detect training anomalies or subsequently compare different training runs.

1.3 Core Contributions

To address the various issues raised in Section 1.2, this M1 internship developed an automated pipeline to construct custom databases based on real-world spatial information and refactored a standard deep learning training framework, thereby laying a solid foundation for future in-depth research on radio map estimation. The main contributions are summarized as follows:

1. **Dataset Construction Pipeline:** Designed and implemented an automated, efficient pipeline for constructing custom radio-map datasets. It caches OpenStreetMap data locally and employs a lazy-loading mechanism to prevent out-of-memory errors associated with large maps; it also supports the downloading of custom tags. Users can configure dataset sample sizes and the scenario size for Sionna simulations. A built-in sliding-window tiling algorithm automatically segments the entire map according to specified parameters.

2. **High-Fidelity Multi-Source HDF5 Dataset:** A dataset comprising samples with 4,000 different spatial characteristics was constructed. The spatial distribution of buildings was extracted from a real OpenStreetMap (OSM) map of the Paris area. The geographical locations and radio-related parameters of the transmitters were obtained from official public records published by the French National Frequency Agency (ANFR). More importantly, the dataset is not stored as loose image files (e.g., (.png))—as was the case with RadioMapSeer—but rather in the more efficient Hierarchical Data Format (.h5). This storage format enables seamless integration with various PyTorch operations (such as data splitting and batch loading) during neural network training, while simultaneously reducing I/O overhead.
3. **Training framework refactoring:** Refactor the training codebase, originally based on procedural programming in Jupyter Notebooks. The refactored training framework comprises several core components: a configuration manager for handling training settings; a trainer for orchestrating training logic; a data loader for fetching samples from datasets and ensuring they meet the model’s input requirements; and a model component for defining the AI model’s architecture. Furthermore, the refactored framework fully replicates the training capabilities of the original codebase and implements RadioUnet as a baseline architecture. These core components provide abstract interfaces, facilitating easy extension to different datasets, model architectures, optimization logic, and more. Additionally, the framework features persistent log management, utilizes TensorBoard for real-time monitoring of metrics and hyperparameters during training, and includes shell script support for submitting training jobs to the LTCI computing cluster via Slurm with a single command.

Below are links to the relevant shared code repositories or websites for your reference:

- Repository for the dataset construction: Dataset Construction Pipeline
- Radio map knowledge base website: Radio Background
- Repository for refactoring training frameworks: Radio Map Prediction

1.4 Report Organization

The remainder of this internship report details the key contributions mentioned in Section 1.3, organized into the following chapters:

- **Chapter 2:** Using the classic RadioUnet method as a case study, this chapter details how deep learning techniques are applied to the task of radio map prediction and explains the rationale behind RadioUnet’s architectural design. Additionally, it introduces the radio-related physics concepts, which may be unfamiliar to researchers with a computer science background, along with a dedicated, simplified knowledge resource website and the Sionna platform used to generate target radio maps during the database construction pipeline. Finally, the chapter analyzes the mathematical formulation of RadioUnet, providing the basis for the training logic implemented when we refactored the training framework.
- **Chapter 3:** This chapter provides a detailed introduction to the principles, functions, and implementation of an automated pipeline for constructing training datasets for radio map prediction tasks. It also presents a comprehensive comparison between the multi-source, high-efficiency dataset (.h5) for Paris, generated using this pipeline, and the classic open-source benchmark dataset RadioMapSeer.

- **Chapter 4:** This chapter outlines the various issues associated with the legacy training codebase, introducing an object-oriented refactoring approach and interface designs that are open to extension. It details the design strategy of decoupling configuration from training logic and discusses the feasibility of deploying training tasks on remote, high-performance clusters. Finally, it presents three designs for experiment record persistence introduced by the new architecture.
- **Chapter 5:** This chapter experimentally validates the primary functions and key features implemented in the preceding two chapters, presenting the details and visualization of the multi-source, high-efficiency database constructed by the pipeline. Additionally, it demonstrates a complete RadioUNet training run using the refactored training framework deployed on the LTCI cluster; a TensorBoard telemetry monitoring interface is provided to confirm the consistency and effectiveness of the refactoring work.
- **Chapter 6:** This chapter summarizes the achievements of the internship and outlines future research directions.

Chapter 2

Technical Background and Related Work

2.1 Radio Map Estimation and Deep Learning

As discussed in Chapter 1, applying artificial intelligence techniques to the task of radio map prediction (RMP)—by modeling it as an image-to-image translation problem—effectively addresses the excessive computational overhead associated with traditional physics-based calculation methods. To gain deeper insight into how AI is integrated into RMP, we examine the classic **RadioUNet** as a representative model; this architecture employs a fully convolutional encoder-decoder structure that exhibits spatial invariance, making it highly compatible with the inherent characteristics of radio maps. During radio wave propagation, the relative spatial trends of signal strength remain consistent regardless of changes in the absolute spatial positions of buildings or transmitters. Consequently, by utilizing convolutions to capture global relative spatial information, the neural network can naturally establish a mapping between these spatial features and the final radio map, thereby achieving results that would otherwise require complex physical computations.

Architecturally, the neural network processes multi-channel spatial tensors via an encoder; during this process, the image dimensions are reduced, and the network’s latent tensors effectively capture macro-level spatial contextual features. Subsequently, a symmetric decoder gradually expands the tensors back to the original image size to generate the final radio map. In the encoder stage, successive downsampling and pooling layers reduce spatial resolution while increasing channel depth to preserve the representational capacity of spatial features, thereby abstracting raw spatial information into high-level latent semantics—such as building density, street orientation, open spaces, and structural occlusion. However, this compression process inevitably results in the loss of structural boundary information regarding buildings, often leading to blurring or smearing artifacts in predictions near edges. To mitigate this issue, the architecture incorporates skip connections, which directly copy high-resolution spatial feature maps from the encoder stage and concatenate them with corresponding layers of the same resolution in the decoder. From the perspective of radio physics, these skip connections supplement the spatial feature input, enabling the decoder to reintroduce sharp boundary gradient information into the training process and thereby reconstruct high-fidelity radio variations around deep urban shadow zones.

For researchers with a background in computer science, grasping the mapping between deep learning and the physical models of electromagnetic propagation—particularly the underlying radio physics—poses a significant challenge. To address this, I have created a simple, publicly accessible resource website that allows anyone to quickly look up relevant physics concepts re-

lated to radio map tasks. Furthermore, recognizing that researchers with a radio background may be less familiar with deep learning model architectures, and also have difficulty to interpret source code, I have provided a detailed breakdown of the classic RadioUnet architecture on the site, accompanied by recommendations for reproducing the model. The site also features a comprehensive overview of the RadioMapSeer dataset (used in the original RadioUnet repository) and includes a quick-reference index. The content covers, but is not limited to, the following topics:

- **Foundational Wireless Communications Principles:** It introduces a range of fundamental concepts and formulas related to wireless communications, including the classic Shannon capacity formula, degrees of freedom (DoF), generalized degrees of freedom (GDoF), cellular wireless network topologies, antenna gain and directivity characteristics, as well as signal transmission mechanisms within Gaussian interference channels subject to signal-to-noise ratio (SNR) constraints.
- **Pathloss:** Path loss can be considered the core of a radio map; indeed, the entire radio map may be viewed as a path-loss field represented as an image. To this end, the website introduces various empirical formulas—ranging from Free-Space Path Loss (FSPL) and general path-loss models to non-deep-learning estimation methods—and explores the mathematical formulations of techniques such as spatial Kriging interpolation, Radial Basis Function (RBF) interpolation, statistical tomography, ray-tracing mechanisms, and the Dominant Path Model (DPM).
- **RadioUnet Architecture and Details:** The specific implementation workflow of the RadioUnet baseline model was analyzed, outlining its primary convolutional modules and functions, as well as key parameters such as tensor preprocessing thresholds, path loss truncation rules, and grayscale normalization operations.
- **Reproducibility Recommendation:** It details practical tips for reproducing results using the original RadioUnet codebase and outlines the structural layout of the RadioMapSeer dataset, enabling future researchers to quickly replicate the work and understand key features of the original repository.

This website played an important role in my subsequent internships, specifically in the development of pipelines for dataset construction and the reconstruction of neural network training framework.

2.2 NVIDIA Sionna Platform

Relying on field measurements to provide the vast amounts of precise signal strength variation data required as training targets for deep learning models is clearly impractical. As analyzed in [3], it is feasible to predict actual signal strength by performing initial training with a large amount of simulated data, followed by fine-tuning using a small amount of real-world measurement data. The NVIDIA Sionna platform offers an excellent solution for generating the necessary large-scale, precise simulated data. Leveraging GPU acceleration, it enables the rapid and efficient generation of high-quality simulation data. In our dataset construction pipeline, Sionna loads the scene files, generated via an automated interface that converts OpenStreetMap (OSM) data into a format directly compatible with Sionna, into actual simulation environments. (This interface was implemented as a non-interactive version based on official Sionna documentation and the Blender plugins mentioned therein.) Appropriate electromagnetic propagation parameters are configured for each object in the scene in accordance with the official Sionna

documentation. During simulation, **Sionna** emits hundreds of rays from transmitters that layerly interact with scene objects [2] to model electromagnetic wave propagation characteristics, ultimately producing the radio map of the entire scene required for our purposes.

Sionna’s underlying hardware interface strategy is the primary reason for its high computational efficiency. Traditional simulation software often relies on serial CPU loops to perform ray-object intersection calculations, resulting in significant computational overhead for complex scenes. In contrast, **Sionna** delivers these intensive computations directly to dedicated NVIDIA RT (Ray Tracing) Cores, leveraging hardware-level algorithmic optimizations to execute complex intersection calculations within the cores themselves in parallel. Consequently, during the radio map matrix generation phase of our dataset construction pipeline, the simulation for a single scene takes only milliseconds. Furthermore, **Sionna** directly outputs the simulated Received Signal Strength (RSS) matrix; this result is highly versatile within the field of radio map and can be easily converted into various standard features commonly used in subsequent deep learning training, such as the logarithmic decibel-milliwatt (dBm) employed in training the RadioUnet baseline model.

2.3 Mathematical Formulation

2.3.1 Generalized Domain and Configurable Tensor Definitions

Let $\mathcal{D} \subset \mathbb{R}^2$ represent the spatial characteristics of the current environment—specifically, the shapes, sizes, and locations of objects within the simulation scene—where a value of 1 indicates the presence of an object that interacts with electromagnetic waves, and 0 indicates its absence. The environment is represented with a resolution of $H \times W$ pixels; this two-dimensional representation is the result of a simplification of spatial information, providing only a top-down view of the scene. However, this 2D representation can be readily extended to 3D by using floating-point values to encode object heights. The same representation scheme can also be applied to characterize the spatial attributes of the transmitter.

Based on the definitions above, the problem of radio map prediction using deep learning can be formulated as the optimization of the parameters θ for a neural network $\mathcal{F}_\theta$. The network takes $X_k \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ as input, representing the spatial characteristics, and outputs an $H \times W$ image in which the value at each pixel location corresponds to the signal strength at that specific point.

To ensure input scalability, the first dimension C_{in} of X_k represents the number of channels; this count can be increased to incorporate additional feature information that influences electromagnetic wave propagation. Typically, the input comprises a 2D building footprint map $B_k \in \{0, 1\}^{H \times W}$ and a 2D transmitter presence matrix $Tx_k \in \{0, 1\}^{H \times W}$; these are the input features provided by the new multi-source, real-world radio map dataset for the Paris region that we constructed in Chapter 3.

Neural network training for radio map prediction tasks is typically supervised. The raw simulation results, serving as targets or labels, cannot be used directly because their wide numerical range often leads to gradient collapse during training. Furthermore, the focus should be on the characteristic variations of significant signals; without proper handling, areas with very weak signals tend to disproportionately influence weight updates and impair learning performance, as downstream radio map tasks are generally less concerned with signal fluctuations in such weak-signal regions. Consequently, it is necessary to preprocess the simulation results (originally in linear power units) by converting them to a logarithmic scale and then normalizing them to

grayscale scalar values $f(x) \in [0, 1]$ through truncation and scaling:

$$f(x) = \max\left(\frac{\psi(x) - \psi_{\text{trnc}}}{\psi_{\text{max}} - \psi_{\text{trnc}}}, 0\right) \quad (2.1)$$

Where $\psi(x)$ is a scalar representing the physical signal strength (e.g., equivalent path loss, dBm field strength values or a linear-watt power); ψ_{trnc} denotes the truncation value, representing the minimum signal strength relevant to radio map tasks; ψ_{max} represents the maximum signal strength in the scenario. This preprocessing of target values avoids misleading gradients arising from noisy regions with excessively weak signals (characteristic of long-tailed distributions) and prevents gradient explosion caused by extreme gaps in absolute signal strength values across locations, thereby ensuring a stable network convergence process.

2.3.2 Configurable Learning Paradigms and Loss Formulations

To accommodate diverse experimental designs, the refactored training framework decouples training process control from internal training logic, enabling the selection of different training methods based on configuration. Specifically, it allows for flexible switching between training methods by selecting different instances that implement the same abstract interfaces for the relevant training logic at runtime.

To maintain compatibility with the original codebase, the refactored framework implements two built-in training logic variants: dense-target training and sparse-target training [3]. The former involves initial training using large amounts of low-precision simulation data, while the latter involves adaptation using minimal high-precision simulation data to simulate the injection of real measurement data. These two training logic variants are described mathematically below.

Paradigm I: Dense-Target Training Logic

When configured for supervised training using dense targets, where every pixel in the scene has a target value, the optimization task during training is to minimize the aggregate error across all pixels in the current scene $\mathcal{D}$. Let K denote the number of independent scenes, and use MSE as the loss function, equivalent to the `torch.nn.MSELoss()` function used by the reconstruction framework:

$$\mathcal{L}_{\text{dense}}(\theta) = \frac{1}{K \cdot |\mathcal{D}|} \sum_{k=1}^K \sum_{x \in \mathcal{D}} \|\mathcal{F}_{\theta}(X_k)(x) - Y_k(x)\|^2 \quad (2.2)$$

The meanings of the specific mathematical symbols in equation (2.2) are as follows:

- θ represents all optimizable parameters in the neural network, such as weights and biases;
- K represents the number of independent scenarios processed in parallel during a single training run.;
- $\mathcal{D}$ represents a single independent scene, where $|\mathcal{D}| = H \times W$ is the total number of pixels;
- $x \in \mathcal{D}$ represents a single pixel in the scene;
- X_k represents the input to the neural network, represents the spatial feature information and the spatial location information of the transmitters in the k -th scene sample;
- $\mathcal{F}_{\theta}(X_k)(x)$ represents the predicted value at pixel x for the k -th input scene sample of the current neural network after forward propagation;
- $Y_k(x)$ represents the target value of the k -th sample at pixel position x .

Paradigm II: Sparse-Target Training Logic

When adapting under the configuration where supervised learning is restricted to a limited, sparse set of pixels, with the remaining pixels lacking valid values, each individual scene k requires an additional mask matrix $M_k \in \{0, 1\}^{H \times W}$ to identify the pixel locations containing valid values, expressed as:

$$M_k(x) = \begin{cases} 1, & \text{if a valid target value is present} \\ 0, & \text{otherwise} \end{cases} \quad (2.3)$$

To ensure that only valid pixel influence the optimization process, the loss function is modified as follows:

$$\mathcal{L}_{\text{sparse}}(\theta) = \frac{1}{K} \sum_{k=1}^K \frac{1}{\|M_k\|_1} \sum_{x \in \mathcal{D}} M_k(x) \cdot \|\mathcal{F}_\theta(X_k)(x) - Y_k(x)\|^2 \quad (2.4)$$

where $\|M_k\|_1 = \sum_{x \in \mathcal{D}} M_k(x)$ represents the number of valid pixels. This formula is mathematically equivalent to the `torch.masked_select` function invoked within the reconstruction framework.

2.3.3 First-Order Optimization

Both training logics described above can be iteratively optimized using first-order stochastic gradient descent algorithms; the reconstructed training framework incorporates the Adam optimizer [7] to optimize the network’s trainable parameters, such as weights and biases:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{\hat{v}^{(t)} + \epsilon}} \hat{m}^{(t)} \quad (2.5)$$

where η denotes the parameterized base learning rate, while $\hat{m}^{(t)}$ and $\hat{v}^{(t)}$ track the first and second moments of the loss gradients.

2.3.4 Physical Meaning of Training Error

Regardless of the configuration, from a mathematical perspective, minimizing the error between predicted and target values reduces prediction uncertainty in the physical world. Since the stage normalization transformation is strictly linear, reducing the mean squared error (MSE) between predicted and target values directly compresses the standard deviation σ of the physical prediction error in the decibel domain. If this error is modeled as a zero-mean Gaussian random variable (i.e., $e_{\text{dB}} \sim \mathcal{N}(0, \sigma^2)$), the parameter σ can be interpreted as an empirical bound on the log-normal shadowing fluctuations of the predicted signal strength [8].

From a practical application perspective, the received signal power must remain above the minimum hardware decoding threshold to maintain a stable connection. Due to inherent statistical prediction errors (σ), the probability that the actual signal fluctuates below this critical threshold—defined as the system outage probability P_{out} —can be mathematically expressed using the standard Gaussian tail Q -function [8, 3]:

$$P_{\text{out}} = 1 - Q\left(\frac{\Gamma_{\text{margin}}}{\sigma}\right) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\frac{\Gamma_{\text{margin}}}{\sigma}} e^{-\frac{u^2}{2}} du \quad (2.6)$$

where $Q(z) = \frac{1}{\sqrt{2\pi}} \int_z^\infty e^{-\frac{u^2}{2}} du$ represents the tail integral of the standard Gaussian distribution, used to calculate the probability that a normally distributed variable exceeds z standard deviations. The term Γ_{margin} represents the difference in decibels between the predicted average

signal strength and the hardware’s minimum reception threshold (i.e., $\Gamma_{\text{margin}} = \text{RSS}_{\text{predicted}} - \text{RSS}_{\text{threshold}}$).

This formula establishes the relationship between neural network error and network efficiency [3]. Minimizing the error standard deviation σ through loss function optimization directly narrows signal uncertainty fluctuations, thereby driving the outage probability P_{out} toward zero. In practical network deployments, a smaller σ implies that operators do not need to reserve excessively conservative power margins to account for unknown signal fluctuations. Consequently, optimizing the model to minimize training error translates directly into tangible physical benefits: under identical hardware constraints, it enables the underlying radio system to safely support higher maximum data throughput and ensures superior network connection reliability [8].

Chapter 3

Scalable Data Pipeline and Dataset Construction

To train a model capable of predicting radio maps within urban environments across multi-source scenarios, the construction of a training dataset that captures authentic urban spatial characteristics and integrates multi-source data is essential. This construction process entails the preprocessing, alignment, and integration of diverse data types—including OpenStreetMap (OSM) data, **Sionna** scene files, and information regarding transmitter locations and radio parameters—with the acquisition of certain data heavily reliant on interactive commercial software. To address this, this chapter presents a scalable, automated, and efficient dataset construction pipeline and demonstrates its application by successfully generating a high-fidelity, multi-source training dataset based on the Paris area.

The pipeline processes unprojected OpenStreetMap (OSM) geospatial data by projecting it onto a local Cartesian coordinate system and reconstructing 3D geometric meshes. It then integrates real-world telecommunications base station data from the French National Frequency Agency (ANFR) and uses **Sionna** to perform simulations—with the resulting outputs serving as training targets—to construct **Paris.h5**, a city-scale, multi-source aggregated radio map dataset. The entire workflow is managed by a decoupled, seven-stage automated script that operates without human intervention.

3.1 Overview of the 7-Stage Architectural Data Pipeline

The constructed pipeline executes a sequence of operations to transform raw, publicly available geospatial data into **Sionna**-compatible scene files and corresponding 2D building outlines; it converts transmitter locations into coordinates within the **Sionna** scene’s planar coordinate system and generates corresponding 2D presence matrices. Simulations are then conducted within the configured **Sionna** scene to obtain a multi-source radio map, which serves as the target value for the dataset. This automated control loop comprises seven independent computational stages:

1. **Stage 1: Region Splitting:** Ingests the filtered ANFR *Cartoradio* database and finds the extreme minimum and maximum coordinates across the target band (eg., 2.6 GHz). These coordinates define the global macro bounding box $\mathcal{B}_{\text{global}}$, which is then divided into a grid of smaller sub-regions, each represented by a metadata descriptor specifying its geometric boundaries. This is to prevent OOM (out of memory) caused by the region being too large.

2. **Stage 2: Per-Block Scene Files Generation & Scene Compilation:** Extracts regional building polygons for each spatial block using a dynamic asset-hashing cache to prevent RAM exhaustion. It executes 3D fan triangulation to convert polygons into binary Polygon File Format (.ply) triangle meshes, while generating XML scene descriptors that map the geometries to specific radio material properties (e.g., concrete permittivity and conductivity) for Sionna ray-tracing.
3. **Stage 3: Building Raster Generation:** Interlaces planar vector geometries with rasterizing utilities, mapping building footprints into dense 2D matrices representing spatial obstacle boundaries.
4. **Stage 4: Transmitter Raster Generation:** Maps the geographic coordinates of transmitters into flat coordinate indexes, generating presence matrices that match the exact spatial dimensions of the obstacle rasters.
5. **Stage 5: Transmitter Relocation:** Runs an automated localized search loop that shifts misplaced base stations falling outside building polygons to the nearest unoccupied rooftop cell. This step corrects spatial coordinate errors present in public recoders.
6. **Stage 6: Per-Block Radio Map Generation:** Loads the serialized XML scene directly into the Sionna, sequentially computes the independent linear Received Signal Strength (RSS) matrices in Watts (W) for each transmitter, and sums them to generate the composite multi-source radio map.
7. **Stage 7: Dataset Consolidation and Merging:** Streams independent block arrays into a centralized layout, stacking the localized 2D matrices along a newly expanded axis and encapsulating the multi-dimensional arrays directly into Hierarchical Data Format 5 (.h5) containers.

3.2 The Sliding-Window Slicing Mechanic

Due to hardware memory limitations, directly processing the entire city map scene is impractical; therefore, we introduce a planar sliding-window tiling algorithm that partitions the entire city scene into uniform tiles based on customizable configurations.

To guarantee metric consistency, the pipeline maintains a strict distinction between the geographic spherical coordinate system and the planar coordinate system used for calculation and storage. The geographical coordinators are maintained in an unprojected coordinate system (EPSG:4326, mapped to the `LocalCRS.OSM.STORAGE` definition, denoted as $\text{SRS}_{\text{storage}}$). However, because sliding-window distance partitioning and ray-tracing simulations require a flat Cartesian coordinate quantified uniformly in meters, geometries are projected into the official French national planar system—the Lambert-93 system (EPSG:2154, mapped to the `LocalCRS.FRANCE.LAMBERT93` configuration, denoted as $\text{SRS}_{\text{local}}$).

The entire urban scenario is represented by $\mathcal{B}_{\text{global}}$, with its minimum and maximum coordinates determined by the minimum and maximum coordinates of the filtered transmitters, ensuring that all transmitters are located within the scenario:

$$\mathcal{B}_{\text{global}} = (\text{lon}_{\min}, \text{lat}_{\min}, \text{lon}_{\max}, \text{lat}_{\max}) \quad (3.1)$$

Let $\mathcal{T}$ denote the coordinate transformation function mapping $\text{SRS}_{\text{storage}} \rightarrow \text{SRS}_{\text{local}}$. The global planar boundary corners, quantified in meters, are evaluated as:

$$(x_{\text{lb}}, y_{\text{lb}}) = \mathcal{T}(\text{lon}_{\min}, \text{lat}_{\min}) \quad (3.2)$$

$$(x_{\text{rt}}, y_{\text{rt}}) = \mathcal{T}(\text{lon}_{\max}, \text{lat}_{\max}) \quad (3.3)$$

To prevent geographic border omissions and index formatting errors during matrix mapping, the pipeline enforces absolute floor and ceiling integer truncation to define the concrete planar domain boundaries:

$$x_{\min} = \lfloor \min(x_{\text{lb}}, x_{\text{rt}}) \rfloor, \quad x_{\max} = \lceil \max(x_{\text{lb}}, x_{\text{rt}}) \rceil \quad (3.4)$$

$$y_{\min} = \lfloor \min(y_{\text{lb}}, y_{\text{rt}}) \rfloor, \quad y_{\max} = \lceil \max(y_{\text{lb}}, y_{\text{rt}}) \rceil \quad (3.5)$$

The total column count N_{cols} and row count N_{rows} comprising the whole city map are calculated using a coverage-first strategy that guarantees entire geographical region capture:

$$N_{\text{cols}} = \left\lceil \frac{(x_{\max} - x_{\min}) - b_m}{s_m} \right\rceil + 1, \quad N_{\text{rows}} = \left\lceil \frac{(y_{\max} - y_{\min}) - b_m}{s_m} \right\rceil + 1 \quad (3.6)$$

where b_m denotes the block size in meters (eg., $b_m = 256$ m) and s_m signifies the sliding stride in meters (eg., $s_m = 128$ m).

Each individual block metadata object (BlockMeta) calculates its geometric bounds directly:

$$x_{\text{start}} = x_{\min} + c \cdot s_m, \quad x_{\text{end}} = x_{\text{start}} + b_m \quad (3.7)$$

$$y_{\text{start}} = y_{\min} + r \cdot s_m, \quad y_{\text{end}} = y_{\text{start}} + b_m \quad (3.8)$$

The pipeline maintains a flexible parameters interface that supports an optional overlap buffer o_m to expand simulation zones if required. For the standard dataset production flow, this buffer is set to $o_m = 0$ m.

To filter out regions that lack simulation utility under the target frequency band, a block is appended to the valid processing queue $\mathcal{M}_{\text{valid}}$ if and only if its geographic area contains at least one valid transmitter derived from the filtered ANFR database:

$$\mathcal{M}_{\text{valid}} = \left\{ \text{BlockMeta}(r, c) \mid |\sigma_{\text{spatial}}(\mathcal{D}_{\text{tx}})| \geq 1 \right\} \quad (3.9)$$

where σ_{spatial} filters transmitters whose longitude and latitude fall within the localized boundaries of the current block.

The execution of this sliding-window partitioning routine is formalized in the algorithm abstraction below:

Algorithm 1 Sliding-Window Sliding Algorithm

```
1: Initialize planar transformation converter:  $\mathcal{T} \leftarrow \text{Transformer}(\text{SRS}_{\text{storage}} \rightarrow \text{SRS}_{\text{local}})$ 
2: Map boundary transmitters coordinates to local planar coordinates:
    $(x_{\text{lb}}, y_{\text{lb}}) \leftarrow \mathcal{T}(\text{lon}_{\text{min}}, \text{lat}_{\text{min}}), \quad (x_{\text{rt}}, y_{\text{rt}}) \leftarrow \mathcal{T}(\text{lon}_{\text{max}}, \text{lat}_{\text{max}})$ 
3: Truncate floating-point values to absolute integer limits to prevent indexing errors:
    $x_{\text{min}} \leftarrow \lfloor \min(x_{\text{lb}}, x_{\text{rt}}) \rfloor, \quad x_{\text{max}} \leftarrow \lceil \max(x_{\text{lb}}, x_{\text{rt}}) \rceil$ 
    $y_{\text{min}} \leftarrow \lfloor \min(y_{\text{lb}}, y_{\text{rt}}) \rfloor, \quad y_{\text{max}} \leftarrow \lceil \max(y_{\text{lb}}, y_{\text{rt}}) \rceil$ 
4: Evaluate global block dimensions using coverage-first:
    $N_{\text{cols}} \leftarrow \lceil ((x_{\text{max}} - x_{\text{min}}) - b_m) / s_m \rceil + 1$ 
    $N_{\text{rows}} \leftarrow \lceil ((y_{\text{max}} - y_{\text{min}}) - b_m) / s_m \rceil + 1$ 
5: for each row index  $r \in [0, N_{\text{rows}} - 1]$  do
6:   for each column index  $c \in [0, N_{\text{cols}} - 1]$  do
7:     Compute planar target block boundaries:
8:      $x_{\text{start}} \leftarrow x_{\text{min}} + c \cdot s_m, \quad x_{\text{end}} \leftarrow x_{\text{start}} + b_m$ 
9:      $y_{\text{start}} \leftarrow y_{\text{min}} + r \cdot s_m, \quad y_{\text{end}} \leftarrow y_{\text{start}} + b_m$ 
10:    Query real-world ANFR transmitters located within block boundaries:
11:     $\mathcal{D}_{\text{local\_tx}} \leftarrow \sigma_{(x \in [x_{\text{start}}, x_{\text{end}}]) \wedge (y \in [y_{\text{start}}, y_{\text{end}}])}(\mathcal{D}_{\text{tx}})$ 
12:    if  $|\mathcal{D}_{\text{local\_tx}}| \geq 1$  then
13:      Construct metadata instance  $\text{BlockMeta}(r, c)$ 
14:      Append  $\text{BlockMeta}(r, c)$  to the valid block list  $\mathcal{M}_{\text{valid}}$ 
15:    end if
16:  end for
17: end for
18: return  $\mathcal{M}_{\text{valid}}$ 
```

3.3 Sionna Scene Files Generation

The scene files loadable by **Sionna** consist of an XML file—defining the electromagnetic properties and rendering colors of objects within the scene—and multiple PLY files representing the spatial characteristics of the 3D objects. Although official documentation suggests using two Blender add-ons to convert OSM data into these scene files, that approach inevitably requires multiple user interactions with the **Blender** graphical interface, failing to meet the requirement for full automation. To address this, we bypassed the **Blender** software itself but—drawing upon the principles of the aforementioned add-ons—implemented a lightweight, simplified conversion process that transforms OSM data directly into **Sionna** scene files.

For each validated block, flat vector object polygons exacted from OSM undergo a dual-attribute lookup pass to dynamically determine concrete their heights. Take objects representing buildings as an example:

$$\mathcal{H}_{\text{building}} = \begin{cases} \alpha, & \text{if 'height' is valid and } \alpha \in \mathbb{R}^+ \\ 3.0 \times \beta, & \text{else if 'building:levels' is valid and } \beta \in \mathbb{R}^+ \\ h_{\text{default}}, & \text{otherwise} \end{cases} \quad (3.10)$$

where h_{default} represents the default height assigned to building objects lacking valid height records (eg., $h_{\text{default}} = 3$ m).

Since the **Sionna** loader only supports .ply files composed of triangular primitives, triangulation is required to decompose arbitrarily shaped, irregular polygons into a unified mesh of interconnected triangles. We employed a lightweight **Fan Triangulation** method, which rapidly constructs surface patches for roofs, grounds, and vertical walls by sequentially connecting a fixed

vertex on the polygon’s surface to all other non-adjacent vertices. Additionally, to handle concave polygons, we utilized the **Ear Cutting** algorithm; specifically, to accommodate scenarios involving polygons with holes, we opted for `mapbox_earcut`, an enhanced implementation of the ear cutting method.

Furthermore, within the official French national projection (Lambert-93), planar coordinates are quantified in absolute numbers exceeding hundreds of thousands of meters. To align the projected coordinates with **Sionna**’s scene coordinates—whose $(0,0,0)$ origin is in the center—the coordinates of all scene objects and transmitters must be relocated. The pipeline automatically calculates the original center point of the current block metadata $(x_{\text{center}}, y_{\text{center}})$ and offsets every vertex position:

$$\begin{aligned} x_{\text{center}} &= \frac{x_{\text{start}} + x_{\text{end}}}{2}, & x_{\text{local}} &= x_{\text{projected}} - x_{\text{center}}, \\ y_{\text{center}} &= \frac{y_{\text{start}} + y_{\text{end}}}{2}, & y_{\text{local}} &= y_{\text{projected}} - y_{\text{center}} \end{aligned} \quad (3.11)$$

This shifting operation ensures that objects and transmitters are correctly located within the current scene.

The **Fan Triangulation** decomposes polygons into separate components using an **earcut**-inspired indexing strategy:

- **Roof Face Generation:** Extracts the polygon perimeter from the earcut output and assigns the building height as the uniform Z coordinate: $V_{\text{roof}} = (x_{\text{local}}, y_{\text{local}}, \mathcal{H}_{\text{building}})$. The `mapbox_earcut` library processes the 2D projected vertices (including holes) and returns a flat index sequence. Roof triangles are constructed by grouping every three consecutive indices from the earcut output: $[i, i + 1, i + 2]$ per triangle, where each index maps to a roof-height vertex in the global vertex array.
- **Ground Face Generation:** For applications requiring a bottom surface, ground triangles would be generated by duplicating the roof vertex positions with $Z = 0$ and reversing the earcut index order for each triangle: $[i + 2, i + 1, i]$, ensuring downward-facing normals.
- **Vertical Wall Generation:** For each boundary edge $(i, i + 1)$ in the perimeter, the corresponding roof vertices at indices i_{roof} and $(i + 1)_{\text{roof}}$ are retrieved via a coordinate-based lookup map. Two **new ground-level vertices** are created for each edge at $(x_i, y_i, 0.0)$ and $(x_{i+1}, y_{i+1}, 0.0)$. Two triangles are then formed to seal the vertical face:

$$[\text{ground}_i, \text{roof}_i, \text{roof}_{i+1}], \quad [\text{ground}_i, \text{roof}_{i+1}, \text{ground}_{i+1}]$$

This per-edge ground vertex creation avoids complex vertex reuse but introduces redundancy in the vertex buffer.

Once the independent objects `.ply` files are compiled, Stage 2 serializes the objects into XML scene file. This step categorizes features into discrete physical layers (`buildings.ply`, `roads.ply`, `railways.ply`, `water.ply`, `forest.ply`, and `terrain.ply`) and binds radio material to specific objects. This structure allows the pipeline to assign electromagnetic properties to different environmental components, according to standard International Telecommunication Union (ITU).

3.4 City Map Rasterization and Transmitter Matrix Generation

To connect to deep learning training, continuous vector boundaries and discrete transmitter coordinate lists must be transformed into matrices. Stages 3 and 4 execute this transformation via rasterization and matrix generation to match the input dimensions of image-to-image models.

Stage 3 converts **OSM** footprints into a 2D matrix. Because the raw object shapes are stored as vector geometric polygons in meters, whereas the deep learning network requires a image-like array, a spatial transformation is required to link pixels to real-world coordinates. The pipeline instantiates a affine transformation matrix $\mathbf{A}$ to serve as a precise positioning scale:

$$\mathbf{A} = \begin{bmatrix} x_{\min} & \Delta_{\text{res}} & 0 \\ y_{\min} & 0 & \Delta_{\text{res}} \end{bmatrix} \quad (3.12)$$

Within this mapping matrix, $x_{\min}$ and $y_{\min}$ lock the anchor origin of the 2D array directly to the specific bottom-left meter coordinate of the block inside the Lambert-93 system. The resolution term Δ_{res} (eg., $\Delta_{\text{res}} = 1.0$ m) dictates that each step of a single pixel row or column represents exactly Δ_{res} meter in the real world.

Guided by the spatial scaling parameters from $\mathbf{A}$, the pipeline executes an automated rasterization operator via the `rasterio` utility. This operator projects the continuous object footprint shapes onto the discrete grid, generating a binary matrix $B_k \in \{0,1\}^{H \times W}$. Grid cells that overlap with any object footprint are filled with a value of 1, while open spaces are filled with 0.

Stage 4 manages transmitter distributions by mapping transmitter coordinates to discrete matrix positions. For each valid transmitter j within a block, its coordinate position ($\text{lon}_j, \text{lat}_j$) is transformed into projected meters (x_j, y_j) via Lambert-93 projection. The pipeline then maps these projected coordinators directly to matrix column indices c_j and row indices r_j :

$$c_j = \left\lfloor \frac{x_j - x_{\min}}{\Delta_{\text{res}}} \right\rfloor, \quad r_j = \left\lfloor \frac{y_j - y_{\min}}{\Delta_{\text{res}}} \right\rfloor \quad (3.13)$$

Using these indexes, the pipeline instantiates a presence matrix $Tx_k \in \{0,1\}^{H \times W}$, setting $Tx_k(r_j, c_j) = 1$ at valid transmitter locations.

In this way, building map and transmitter locations can align with neural network inputs.

3.5 Transmitter Relocation

A significant challenge encountered when processing real-world public records is coordinate drift. This drift causes transmitters to map incorrectly to open spaces instead of building roofs, violating physical deployment logic and introducing error into ray-tracing simulations.

To resolve this problem, Stage 5 introduces a spatial correction step that re-aligns misplaced transmitters with nearby buildings. For each active base station located at matrix coordinates (r_j, c_j) : If $B_k(r_j, c_j) = 1$, this transmitter is confirmed to sit on an existing building, and its coordinates are preserved. However, if $B_k(r_j, c_j) = 0$, a localized spatial correction searches a square neighborhood bounded by a search range parameter d (eg., $d = \pm 20$ m):

$$\Omega(r_j, c_j) = \left\{ (r, c) \mid |r - r_j| \leq d \wedge |c - c_j| \leq d \wedge 0 \leq r < H \wedge 0 \leq c < W \right\} \quad (3.14)$$

The nearest valid building cell that is not already occupied by another antenna structure is found:

$$(r^*, c^*) = \arg \min_{(r, c) \in \Omega(r_j, c_j)} [(r - r_j)^2 + (c - c_j)^2] \quad (3.15)$$

$$\text{subject to } B_k(r, c) = 1 \quad \wedge \quad Tx_k(r, c) \neq 1 \quad (3.16)$$

If a valid destination cell (r^*, c^*) exists, the transmitter is moved to the corrected position by setting $Tx_k(r_j, c_j) = 0$ and $Tx_k(r^*, c^*) = 1$. If multiple candidate cells share the same minimal distance, ties are broken by selecting the cell with the lowest row index followed by the lowest column index. If no building structures are found within the search window Ω , the transmitter position is left unchanged. This spatial re-alignment step ensures that subsequent simulations conform to realistic transmitter deployment constraints.

3.6 Multi-Source Radio Map Matrix Generation

Stage 6 computes the aggregate multi-source radio map by accumulating the power from all valid transmitters within the current block. To simplify the analytical complexity and avoid tracking intractable carrier-level phase variations, the aggregation pipeline operates under a non-coherent power superposition assumption. This analytical simplification is physically grounded by the fact that co-existing real-world transmitters operate independently without synchronized fixed phase relationships. Therefore, instead of modeling complex constructive or destructive wave interference patterns, the total received signal strength (RSS) at any spatial pixel can be accurately calculated via a straightforward linear summation of the individual signal powers.

To model this correctly, the pipeline queries the low-level output from the **Sionna** ray-tracing, which yields scalar matrices quantified in linear power scales of Watts (W). For a block containing J valid transmitters, the aggregate field $\mathbf{P}_{\text{core}} \in \mathbb{R}^{H \times W}$ is computed via a cell-wise matrix addition:

$$\mathbf{P}(x) = \sum_{j=1}^J \mathbf{P}_j(x), \quad \forall x \quad (3.17)$$

where $\mathbf{P}_j(x)$ represents the independent linear power matrix (in Watts) simulated for the j -th transmitter.

While deep neural networks generally require input targets to be represented in the logarithmic decibel-milliwatt (dBm) scale to compress the wide value range and stabilize gradient convergence during training, storing the raw data directly in dBm would complicate multi-source superposition. Because converting linear power in Watts to a logarithmic scale is a computationally trivial mathematical transformation, this conversion can be easily executed on-the-fly during the deep learning data loading pre-processing phase. Consequently, to maintain physical consistency and raw data integrity, the final radio map matrices are preserved using the linear power scale of Watts (W).

3.7 Comparison with RadioMapSeer

The final phase of the pipeline, Stage 7, aggregates all independent block arrays into a unified data structure.

Using this pipeline, we successfully constructed a multi-source, high-fidelity, and efficient dataset based on spatial characteristics and public transmitter records from the Paris region. To highlight its architectural improvements and data diversity, we compare it against the classic **RadioMapSeer** benchmark dataset.

Table 3.1: Comparison between RadioMapSeer and the Proposed Dataset

Technical Dimensions	RadioMapSeer [3]	Proposed Dataset (Ours)
BS Distribution	Randomized synthetic grid scattering	Real Cartoradio infrastructure (ANFR)
Total Unique Maps	700	> 4,000
Transmitter Model	Single-Source	Multi-Source
File Layout	Loose independent .png and .json files	Monolithic hierarchical HDF5 store
Disk I/O Paradigm	Scattered random reads (I/O bound)	Contiguous streaming block memory allocation

As detailed in Table 3.1, the proposed dataset introduces significant improvements in both

data realism and computational I/O performance. In details, **RadioMapSeer** stores its data as an unmanaged collection of thousands of loose `.png` image layers accompanied by detached `.json` metadata files. During iterative deep learning training loops, this structure forces PyTorch dataloaders to perform scattered random read requests across the operating system file system. This layout creates severe random I/O bottlenecks, which wastes CPU cycles and leaves high-performance GPUs wait for data. To resolve this bottleneck, the proposed dataset utilizes the Hierarchical Data Format (`.h5`). It stacks individual block matrices into a unified, monolithic array structure on disk:

$$\mathbf{T}_{\text{buildings}} \in \mathbb{N}_0^{N \times H \times W}, \quad \mathbf{T}_{\text{transmitters}} \in \mathbb{N}_0^{N \times H \times W}, \quad \mathbf{T}_{\text{radiomap}} \in \mathbb{R}^{N \times H \times W} \quad (3.18)$$

Chapter 4

Training Framework Refactoring

Although Chapter 3 established a scalable, automated pipeline for generating multi-source datasets, an easily maintainable and extensible software training framework is essential to effectively utilize these datasets for training neural network models. This chapter details the process of refactoring the original procedural codebase into a decoupled, object-oriented PyTorch framework.

4.1 Legacy Code Evaluation and Technical Debt

The prototype codebase originally developed for the baseline model relied heavily on interactive Jupyter Notebooks (`.ipynb`). While Notebooks are highly useful for early exploratory analysis and rapid prototyping, using them to maintain a stable and scalable training codebase presents numerous issues:

- **Control Flow and Code Redundancy:** The legacy codebase lacked standard modular abstractions, causing separate logical stages to be interleaved within sequential notebook cells. To handle multiple training modes, the implementation relied on large, hard-coded internal conditional branches (`if-else` blocks) inside the main execution loops. This design coupled code configuration directly with training process logic, resulting in redundant, duplicated iteration boilerplate that complicated baseline maintenance.
- **Implicit Volatility via Sequential Execution:** Because the execution flow depended on the manual, order-dependent execution of notebook cells, the runtime is highly fragile. Variables, architecture instances, and pointers were repeatedly overwritten in place within the same scope to transition between different training configurations. This makes the code highly vulnerable to execution order errors.

4.2 Modular Object-Oriented Framework Design

To resolve these problems, the legacy codebase was completely refactored into a modular, object-oriented software framework:

1. **Model:** Encapsulates the neural network architectural logic. It isolates the model definitions from others, exposing standardized input-output tensor interfaces and providing pre-integrated baselines like `RadioUNet` while remaining open for future architectural extensions.

2. **Trainer:** Governs the core optimization loops, background execution, and model checkpointing routines, implementing standard training lifecycles while remaining independent of specific dataset parameters.
3. **Data Manager:** Divided into two layers: a **Dataset** layer handling data loading mechanics, and a **DataModule** layer that manages dataset splitting logic, replacing the manual slicing indices of the legacy codebase.
4. **Config:** Controls the parsing and validation of external configuration files, ensuring that all runtime variables conform to expected formats before execution begins.

To highlight the practical benefits of this refactoring, Table 4.1 contrasts the structural characteristics of the original legacy setup against the current object-oriented solution.

Table 4.1: Comparison between Legacy and Refactored Training Frameworks

Comparison	Original Solution	Current Solution
Parameter Management	A large number of hard-coded parameters scattered across various modules.	Unified YAML configuration file driver for easy experimental adjustments.
Code Structure	High functional repetition and tightly coupled training loops.	High core component reuse, making future expansions and modifications simpler.
Process Monitoring	Manual console printing outputs.	TensorBoard curve monitoring for loss and hyperparameter.
Training Platform	Interactive notebooks requiring manual configurations and triggers.	Headless one-click training compatible with local environments or remote HPC clusters.

4.2.1 Template Method Pattern and Subclass Hooks

The architectural core of the new **Trainer** module utilizes the *Template Method design pattern* to standardize the training lifecycle while preserving algorithmic flexibility. The abstract base **BaseTrainer** class defines the invariant skeleton of the training process through its two main methods: `fit` for training and validation, and `test` for evaluation. These two main functions control the high-level main flow, ensuring that epoch iterations, learning rate updates, early stopping, and metric tracking follow a strict and consistent order across all experiments.

To allow customization without modifying this core flow, the base class sends specific setup and computation steps to downstream subclasses through dedicated virtual hooks. These hooks are divided into two distinct levels:

- **Initialization-Level Hooks (Init-Level):** Executed during instance creation to set up baseline objects. Methods such as `_set_logger`, `_set_criterion`, and `_set_optimizer` provide sensible defaults (such as Adam optimization and MSE loss), but allow subclasses to override them to inject alternative optimization parameters easily.
- **Iteration-Level Hooks (Step-Level):** Executed during the data loop to handle single-batch calculations. Subclasses must implement `_train_step`, `_val_step`, and `_test_step` to define how inputs are processed by the network and how raw outputs are converted into backpropagation loss.

The refactoring codebase natively provides two built-in subclasses—**UnmaskedTrainer** and **MaskedTrainer**—to fully satisfy the training workflows of the legacy codebase. These pre-implemented components directly replicate the original dense case and masked sparse case calculations without duplicating any redundant functions, and ensures strict functional alignment with the mathematical formulations established in Section 2.3.

4.3 YAML-Driven Configuration and Cluster Deployment

To decouple config parameters from execution code, the refactored framework uses an external configuration layer driven by structured YAML files. This centralized configuration file manages all active variables:

- **Global Reproducibility Anchor (seed):** Sets a fixed global random seed to ensure absolute experimental reproducibility across distinct execution platforms.
- **Data Apportionment and Loading (load):** Governs the partition ratios for splitting dataset and manages batch size for different execution phases.
- **Train Control (train):** Controls the baseline learning rates, total epoch limits, optimizer paramater and **EarlyStopping** logic.
- **Environment and File Path (data):** It manages file paths for diverse simulation configurations, toggles urban variations like vehicle noise interference or missing building counts, and controls the signal clipping threshold. Additionally, it can dynamically informs the dataloader whether to generate a binary mask matrix, switching the framework between the **MaskedTrainer** and **UnmaskedTrainer** modes.

During initialization, the **Config** module parses the target `.yaml` file and runs an internal validation pass to verify data formats.

This programmatic decoupling removes interactive GUI constraints, enabling seamless headless deployment on remote High-Performance Computing (HPC) clusters. To establish a standardized and stable runtime environment across distinct computing nodes, a dedicated Apptainer definition file (`.def`) was provided. This containerization strategy guarantees complete environment consistency and hardware-level compatibility regardless of the host cluster configuration.

Governed by this containerized setup, the entire training pipeline can be dispatched to remote compute nodes via the **Slurm** workload manager, executing the master routine inside the active container environment using a single entry command. It transforms the unmanaged, cell-dependent legacy workflows into a stable, automatic, highly scalable scientific computing framework

4.4 Experiment Tracking

When moving training operations to headless remote servers, maintaining visibility into model convergence requires a robust tracking system. The refactored framework replaces basic manual console logging with a complete experiment monitoring solution divided into some components, which we will describe below in details.

4.4.1 TensorBoard Monitoring

The framework integrates TensorBoard to transition tracking away from loose terminal output into a persistent and visual tracking repository. By emitting scalar matrices at regular

batch intervals, the system generates real-time curves for loss values across both training and validation sets. Monitoring these synchronized curves helps researchers quickly diagnose learning issues—such as training anomalies, slow convergence, or early signs of overfitting—without requirement to manually inspect raw numerical tables.

4.4.2 Log File Management

Independent of the visual interface, the framework features an automated file log manager. Every experimental run generates a unique, timestamped session directory within a centralized `/logs` repository. This directory stores a complete textual history of the training run. This isolated storage prevents logs from overlapping, ensuring that an accurate historical record is preserved for every background run.

4.4.3 Automated Experiment Result Persistence

At the end of the training lifecycle, the core execution metrics, structural hyper-parameters, and performance statistics are automatically written into a structured registry file. To provide a concrete example, Table 4.2 illustrates an extracted slice of this automated registry, capturing the comparative metrics across distinct architectural and dataset configurations.

Table 4.2: Example of the Automated Experiment Registry Log

Architecture	Dataset Description	RMSE	NMSE	Time/Sample	Recording Time
RadioUNet	DPM No Cars Complete Map	0.0412	0.0310	4.24 ms	2026-05-26 13:23
RadioWNet	IRT4 (300) No Cars Complete Map	0.0389	0.0276	2.65 ms	2026-05-26 13:40

This automated persistence system enables efficient historical tracking. Instead of forcing researchers to manually open and parse individual log files across dozens of background experimental runs, the registry provides a centralized, tabular dashboard. This allows for immediate empirical comparison of network architecture, scaling behaviors, and simulation variants, accelerating the experimental lifecycle.

Chapter 5

Experimental Evaluation and Functional Verification

This chapter provides an experimental validation of the core deliverables produced during this internship: the dataset construction pipeline with its newly constructed multi-source product and the refactored deep learning framework.

Before scaling up to training on the new, broader scale, and complex environmental dataset, it is crucial to verify that the training framework’s components function correctly in practice and the format and shape of the new dataset. To achieve this, the experimental evaluation is divided into two passes: first, a validation to inspect the spatial alignment of the constructed dataset within the pipeline mentioned in Chapter 3; and second, a benchmarking run to check the training logic and convergence behavior of the refactored framework on the LTCI computing cluster.

5.1 Dataset Validation

Before injecting the board scal, real city-case, new dataset into the neural training framework, we need firstly to evaluate the geometric dimensions and internal specifications of this dataset. The automated data pipeline developed in Chapter 3 stores each urban simulation block as a multi-channel matrix. The exact mapping layout of the dataset is systematically cataloged in Table 5.1. We use the dataset comprising real-world spatial data from the Paris region and multiple transmitters in this region, constructed by the above pipeline, as an example for our analysis.

Table 5.1: Dataset Schema

Channel Layer	Physical Significations	Data Type	Dimensions	Value Bounds
B_k (Layer 0)	Morphological Building Mask	Binary Integer	256×256	$\{0, 1\}$
Tx_k (Layer 1)	Relocated Multi-Source Transmitter Presence	Binary Integer	256×256	$\{0, 1\}$
RSS_k (Target)	Aggregate Linear Received Signal Strength (RSS) in Watts	Float32	256×256	$[0, RSS_{\max}]$

To visually evaluate the spatial correctness and mathematical alignment guaranteed by the pipeline, a representative urban sample slice is decomposed. Figure 5.1 presents a 2×2 visualization that illustrates the structural decomposition of the input channels alongside the target

simulation and the final multi-channel superimposed alignment schematic.

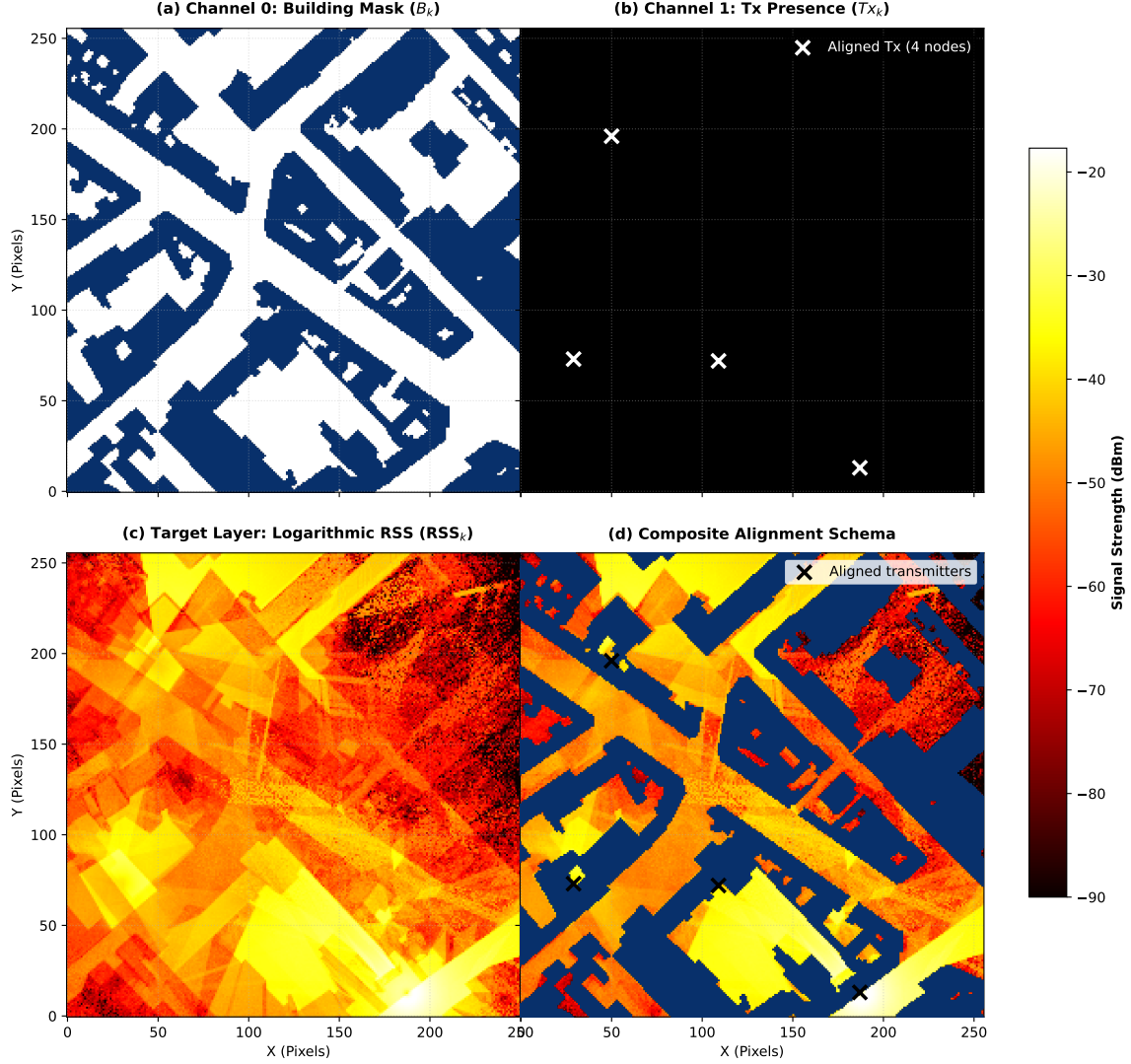


Figure 5.1: multi-channel decomposition and superimposed alignment of a sample in dataset: (a) Channel 0: Morphological building obstacle mask B_k ; (b) Channel 1: Relocated multi-source base station presence matrix Tx_k ; (c) Target layer: Aggregate linear received signal strength RSS_k represented in the logarithmic dBm; (d) Composite alignment schema illustrating the simultaneous superposition of the building mask, the aligned transmitter directly over the target layer

By executing the automated neighborhood search loop, the drifted transmitter is dynamically relocated onto the building top. To verify the effectiveness, Figure 5.2 isolates and magnifies the transmitter alignment over the target rooftop layout.

As visually demonstrated in Figure 5.2, the relocation process effectively solves the positional drift. This adjustment ensures that simulation rays propagate from physically authentic transmitter locations rather than drifting open coordinates. Consequently, this unified verification confirms that the dataset’s data layers achieve complete spatial modality alignment, establishing a high-fidelity and mathematically rigorous foundation for subsequent deep learning model optimizations.

Figure 5.3 displays the buildings, roads, trees, and transmitters in the 14th arrondissement

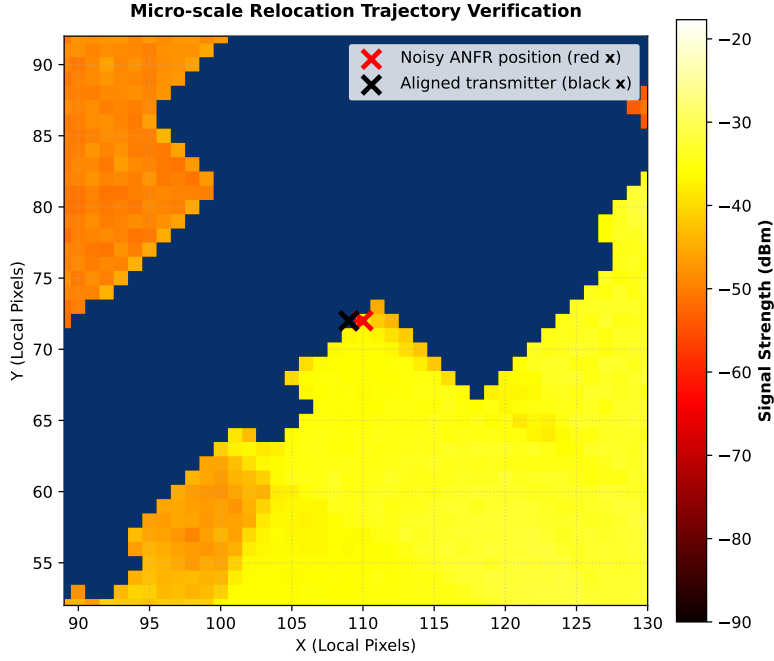


Figure 5.2: Visualization of the automated neighborhood search loop: the uncorrected transmitter (red $\times$) drifting onto open space due to public ANFR record noise is successfully pulled back and relocated onto the building (black $\times$).

of Paris, displayed using the built-in `preview` function in Sionna.

5.2 Experimental Training Dynamics and Record Validation

To verify the complete integration of the modular software architecture and the template method pattern components, the training loop was deployed inside an `Apptainer` container on a remote high-performance computing (HPC) server cluster. The global runtime parameters were driven entirely via an externalized `config.yaml` script, decoupling execution logic from state configurations. To guarantee a standardized functional baseline, the framework was executed under the `UnmaskedTrainer` dense case using the standard `RadioMapSeer` benchmark dataset.

The optimization tracking records were streamed directly into a persistent `TensorBoard` repository at regular epoch intervals. Figure 5.4 illustrates the loss decay curves captured during the baseline functional verification pass.

As demonstrated by the synchronized curves in Figure 5.4, the refactored framework exhibits a robust and stable non-convex convergence behavior. Throughout the entire baseline trajectory, the headless computational execution completed smoothly on the remote cluster nodes, without triggering any runtime crashes.

The successful execution of these validation experiments that the custom abstract hooks function and other refactoring features in strict alignment with the software specifications detailed in Chapter 4. This functional validation demonstrates that the refactoring training framework establishes a stable, automatic, and consistent with the original codebase infrastructure capable of supporting future complex deep learning experiments.



Figure 5.3: Sionna Simulation Preview

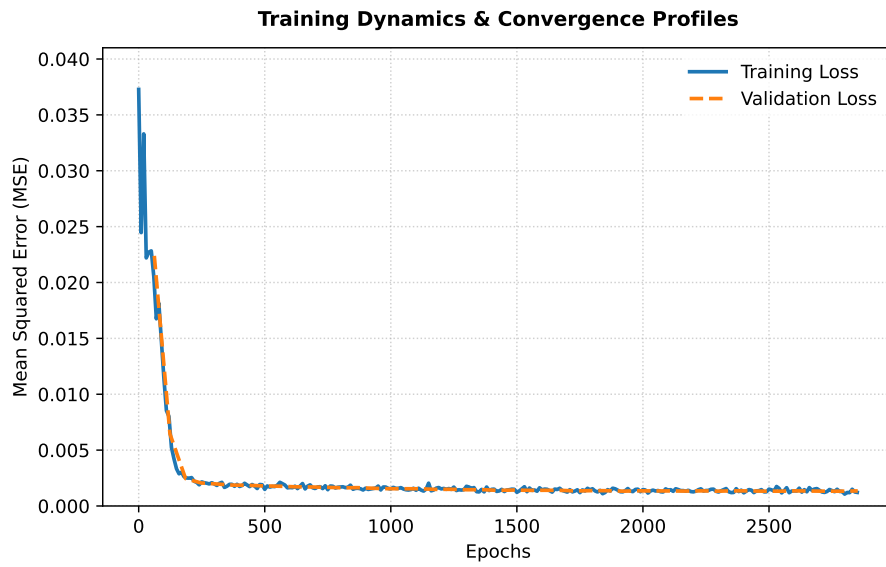


Figure 5.4: TensorBoard monitoring showing the loss decay curves for both training and validation sets under dense MSE regression optimization.

Chapter 6

Conclusion and Future Work

6.1 Summary of Internship Achievements

This internship has successfully developed and delivered a scalable dataset construction pipeline and a modular deep learning training framework. The major physical achievements completed during this internship are summarized as follows:

- **High-Fidelity Multi-Source Dataset Production:** Break through the conventional single-source simulation limit by implementing an automated dataset construction pipeline. Utilizing authentic transmitter records from the French national frequency agency (ANFR) and the NVIDIA Sionna ray-tracing platform to compile a new multi-source dataset. This dataset contains over 4,000 unique multi-source samples that natively model non-coherent linear power aggregation over real city scene.
- **Object-Oriented Training Framework Refactoring:** Migrate the unmanaged, cell-by-cell procedural codebase from legacy Jupyter Notebooks into a decoupled, object-oriented PyTorch framework. Introduce an externalized YAML configuration layer and integrate TensorBoard telemetry visualization to decouple the execution logic from run-time setting and log management. This clean-up eliminates inline hard-coding and enables headless, single-command execution on high-performance remote GPU computing clusters.

6.2 Future Research Outlook

Building upon the foundation established in this internship, there are two coherent dimensions can be further extended: enriching data input modalities and transitioning the optimization paradigm from empirical fitting to physically consistent modeling.

- **Multi-channel Environmental Matrix:** While the current framework demonstrates stable convergence using 2D building binary masks, the deep neural network can be expanded by enriching the input data dimensionality. Future works can extend the input data from a single mask into a multi-channel geometric structure. First, building height data can be formatted as a continuous height-map channel, allowing the network to explicitly capture the 3D scene spatial characteristics. Second, different objects from OpenStreetMap (OSM) (such as rivers, roads or forests) can be integrated as different mask matrix. Take rivers as an example, the water surfaces of rivers provide low-attenuation specular reflections rather than heavy penetration losses. This channel will guide the model to capture unique propagation characteristics across river corridors. Finally, the 3D spatial profiles

of the transmitters (including heights and directional orientations) can be encoded as an additional continuous spatial channel.

- **Physics-Informed Neural Network Architecture:** Benefiting from the thoroughly restructured and modularized training framework, the immediate next step is to execute scalable training runs on the newly compiled database. Moreover, to transition the framework away from a pure statistical black-box, future architectural iterations can implement a Physics-Informed Neural Network (PINN) architecture. Instead of forcing the network to learn electromagnetic wave propagation entirely from scratch, we propose to inject physical domain knowledge directly into the pipeline through two parallel strategies. First, on the input side, a computationally cheap analytical propagation model (such as Free-Space Path Loss, FSPL) can be executed on-the-fly to generate a baseline field, which is concatenated as an explicit prior channel to provide a physical anchor. Second, on the optimization side, a custom physics-driven loss term can be formulated to penalize non-physical predictions, such as unrealistic signal penetrations inside building regions. By optimizing a joint loss function ($\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda\mathcal{L}_{\text{physics}}$), this setup will compress the gradient boundaries and significantly accelerate training convergence.

Bibliography

- [1] Sébastien Faye, Ramiro Camino, Ghaya Rzig, Peiman Alipour Sarvari, Neamah Al-Naffakh, Juan Carlos Estrada-Jiménez, Enric Pardo, and Djamel Khadraoui. A survey on emf-aware mobile network planning. *IEEE Access*, 11:85927–85950, 2023.
- [2] Ingo Wald, Solomon Boulos, and Peter Shirley. Ray tracing animated scenes using bounding volume hierarchies. *ACM Transactions on Graphics*, 2006.
- [3] Ron Levie, Çağkan Yapar, Gitta Kutyniok, and Giuseppe Caire. Radiounet: Fast radio map estimation with convolutional neural networks. *IEEE Transactions on Wireless Communications*, 20(6):4001–4015, 2021.
- [4] Bin Feng, Meng Zheng, Wei Liang, and Lei Zhang. A recent survey on radio map estimation methods for wireless networks. *Electronics*, 14(8), 2025.
- [5] Adriana Fernandes Veludo, Bram Stroobandt, Han Van Bladel, Nekane Sandoval-Diez, Kenneth Deprez, Sam Aerts, Wassim Ben Chikha, Joe Wiart, Zsuzsanna Vecsei, Péter Pál Necz, György Thuróczy, Martina Benini, Marta Bonato, Silvia Gallucci, Gabriella Tognola, Marta Parazzini, Lea Belácková, Nina Vaupotič, Pawel Mamrot, Magda Marianska, Piotr Poltanski, Kinga Polanska, Matthew Stamets, Patricia de Llobet, Gemma Castaño-Vinyals, Mònica Guxens, Paige M. Hulls, Frank de Vocht, Wout Joseph, and Martin Röösl. Assessing radiofrequency electromagnetic field exposure in multiple microenvironments across ten european countries with a focus on 5g. *Environment International*, 200:109540, 2025.
- [6] International Commission on Non-Ionizing Radiation Protection. Guidelines for limiting exposure to electromagnetic fields (100 khz to 300 ghz). *Health Physics*, 118(5):483–524, 2020.
- [7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [8] Andrea Goldsmith. *Wireless communications*. Cambridge university press, 2005.